Data Structures - Homework 3

Q1. Write a C program to store n elements in an array using dynamic memory allocation and find the largest number.

Code:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n, *a, large, i;
    printf("Enter n: ");
    scanf("%d", &n);
    a = (int*)malloc(n * sizeof(int));
    printf("Enter Elements: ");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    large = a[0];
    for(i = 1; i < n; i++)
    {
        if(a[i] > large)
        {
            large = a[i];
        }
    }
    printf("Largest = %d\n", large);
    free(a);
    return 0;
}
```

Output:

```
Enter n: 5
Enter Elements: 10 20 30 40 50
Largest = 50
=== Code Execution Successful ===
```

Q2. Print an array in reverse order using pointers.

Code:

```
#include <stdio.h>
int main()
{
    int a[100], n, i;
    int *p;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter Elements: ");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    p = &a[n - 1];
    printf("Reverse order:\n");
    for(i = n - 1; i >= 0; i--)
```

```
    {
        printf("%d ", *p);
        p--;
    }
    return 0;
}
```

Output:

```
Enter n: 3
Enter Elements: 1 2 3
Reverse order:
3 2 1
=== Code Execution Successful ===
```

Q3. Write a C program to store info of students using structure. Sort them by Name.

Code:

```
#include <stdio.h>
#include <string.h>
struct Student
{
    int roll;
    char name[50];
    float marks;
};
int main()
{
    struct Student s[100], temp;
    int n, i, j;
    printf("Enter no. of students: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter roll, name, and marks: ");
        scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);
    }
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(strcmp(s[i].name, s[j].name) > 0)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }
    printf("\nStudents sorted by name:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d %s %.2f\n", s[i].roll, s[i].name, s[i].marks);
    }
    return 0;
}
```

Output:

```
Enter no. of students: 2
```

```
Enter roll, name, and marks: 20 sri 80
Enter roll, name, and marks: 100 Ace 90

Students sorted by name:
100 Ace 90.00
20 sri 80.00
=== Code Execution Successful ===
```

Q4. Write a C program to add 'n' distances given in inch-feet system using structures.

Code:

```
#include <stdio.h>
struct Distance
{
    int feet;
    float inch;
};
int main()
{
    struct Distance d, sum = {0, 0.0};
    int n, i;
    printf("Enter no. of distances: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter feet and inches: ");
        scanf("%d %f", &d.feet, &d.inch);
        sum.feet += d.feet;
        sum.inch += d.inch;
    }
    sum.feet += (int)(sum.inch / 12);
    sum.inch = (int)sum.inch % 12 + (sum.inch - (int)sum.inch);
    printf("Total distance = %d feet %.2f inches\n", sum.feet, sum.inch);
    return 0;
}
```

Output:

```
Enter no. of distances: 2
Enter feet and inches: 8 6
Enter feet and inches: 1 7
Total distance = 10 feet 1.00 inches
=== Code Execution Successful ===
```

Q5. Write a C program to add 2 complex numbers by passing structure to a function.

Code:

```
#include <stdio.h>
struct Complex
{
    float real;
    float img;
};
struct Complex add(struct Complex c1, struct Complex c2)
{
    struct Complex res;
```

```
    res.real = c1.real + c2.real;
    res.img = c1.img + c2.img;
    return res;
}
int main()
{
    struct Complex c1, c2, sum;
    printf("Enter real and img of 1st no: ");
    scanf("%f %f", &c1.real, &c1.img);
    printf("Enter real and img of 2nd no: ");
    scanf("%f %f", &c2.real, &c2.img);
    sum = add(c1, c2);
    printf("Sum = %.1f + %.1fi\n", sum.real, sum.img);
    return 0;
}
```

Output:

```
Enter real and img of 1st no: 1 2
Enter real and img of 2nd no: 2 1
Sum = 3.0 + 3.0i
=== Code Execution Successful ===
```

Q6. Write a C program to maintain inventory management system using structure.

Code:

```
#include <stdio.h>
struct Product
{
    int id;
    char name[50];
    int qty;
    float price;
};
int main()
{
    struct Product p[100];
    int n, i;
    printf("Enter no. of products: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter ID, Name, Qty, Price: ");
        scanf("%d %s %d %f", &p[i].id, p[i].name, &p[i].qty, &p[i].price);
    }
    printf("\nInventory Details:\n");
    for(i = 0; i < n; i++)
    {
        printf("ID: %d\nName: %s\nQuantity: %d\nPrice: %.2f\n\n",
               p[i].id, p[i].name, p[i].qty, p[i].price);
    }
    return 0;
}
```

Output:

```
Enter no. of products: 2
Enter ID, Name, Qty, Price: 101 Soap 2 30
Enter ID, Name, Qty, Price: 301 Oil 1 100

Inventory Details:
```

```
ID: 101
Name: Soap
Quantity: 2
Price: 30.00

ID: 301
Name: Oil
Quantity: 1
Price: 100.00

=== Code Execution Successful ===
```

Q7. Write a C program for decimal to binary, octal, and hexadecimal conversion.

Code:

```
#include <stdio.h>
int main()
{
    int n, temp, rem;
    int binary[32], i = 0, j;
    printf("Enter decimal no: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {
        rem = temp % 2;
        binary[i] = rem;
        i++;
        temp = temp / 2;
    }
    printf("Binary = ");
    for(j = i - 1; j >= 0; j--)
    {
        printf("%d", binary[j]);
    }
    printf("\nOctal = %o\n", n);
    printf("Hexadecimal = %X\n", n);
    return 0;
}
```

Output:

```
Enter decimal no: 25
Binary = 11001
Octal = 31
Hexadecimal = 19
=== Code Execution Successful ===
```

Q8. Write a C program to calculate difference between two time periods.

Code:

```
#include <stdio.h>
struct Time
{
    int hours;
    int mins;
    int sec;
};
```

```
int main()
{
    struct Time t1, t2, diff;
    printf("Enter 1st time (h m s): ");
    scanf("%d %d %d", &t1.hours, &t1.mins, &t1.sec);
    printf("Enter 2nd time (h m s): ");
    scanf("%d %d %d", &t2.hours, &t2.mins, &t2.sec);
    if(t2.sec < t1.sec)
    {
        t2.sec += 60;
        t2.mins--;
    }
    if(t2.mins < t1.mins)
    {
        t2.mins += 60;
        t2.hours--;
    }
    diff.sec = t2.sec - t1.sec;
    diff.mins = t2.mins - t1.mins;
    diff.hours = t2.hours - t1.hours;
    printf("Difference: %02d:%02d:%02d\n", diff.hours, diff.mins, diff.sec);
    return 0;
}
```

Output:

```
Enter 1st time (h m s): 10 20 30
Enter 2nd time (h m s): 12 45 50
Difference: 02:25:20
=== Code Execution Successful ===
```

Q9. Write a C program to implement an employee management system using structures.

Code:

```
#include <stdio.h>
struct Employee
{
    int id;
    char name[50];
    float sal;
};
int main()
{
    struct Employee e[100];
    int n, i;
    printf("Enter no. of employees: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter Emp ID, Name, Salary: ");
        scanf("%d %s %f", &e[i].id, e[i].name, &e[i].sal);
    }
    printf("\nEmployee Details:\n");
    for(i = 0; i < n; i++)
    {
        printf("ID: %d\nName: %s\nSalary: %.2f\n\n",
               e[i].id, e[i].name, e[i].sal);
    }
    return 0;
```

```
}
```

Output:

```
Enter no. of employees: 2
Enter Emp ID, Name, Salary: 1001 Zoe 20000
Enter Emp ID, Name, Salary: 4001 Law 20000

Employee Details:
ID: 1001
Name: Zoe
Salary: 20000.00

ID: 4001
Name: Law
Salary: 20000.00

=== Code Execution Successful ===
```

Q10. Write a C program to calculate area and perimeter of rectangle and triangle using function pointers.

Code:

```
#include <stdio.h>
float rec_area(float l, float b)
{
    return l * b;
}
float rec_peri(float l, float b)
{
    return 2 * (l + b);
}
float tri_area(float b, float h)
{
    return 0.5 * b * h;
}
float tri_peri(float a, float b, float c)
{
    return a + b + c;
}
int main()
{
    float l, b, h, s1, s2, s3;
    float (*calc2)(float, float);
    float (*calc3)(float, float, float);

    printf("Enter length and breadth of rectangle: ");
    scanf("%f %f", &l, &b);
    calc2 = rec_area;
    printf("Rectangle Area = %.2f\n", calc2(l, b));
    calc2 = rec_peri;
    printf("Rectangle Perimeter = %.2f\n", calc2(l, b));

    printf("\nEnter 3 sides of triangle: ");
    scanf("%f %f %f", &s1, &s2, &s3);
    printf("Enter base and height: ");
    scanf("%f %f", &b, &h);
    calc2 = tri_area;
    printf("Triangle Area = %.2f\n", calc2(b, h));
    calc3 = tri_peri;
```

```
    printf("Triangle Perimeter = %.2f\n", calc3(s1, s2, s3));
    return 0;
}
```

Output:

```
Enter length and breadth of rectangle: 10 5
Rectangle Area = 50.00
Rectangle Perimeter = 30.00

Enter 3 sides of triangle: 3 4 5
Enter base and height: 3 4
Triangle Area = 6.00
Triangle Perimeter = 12.00
=== Code Execution Successful ===
```